

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Математические основы интеллектуальных систем»

За 3 семестр

Тема: «Нахождение кратчайшего пути в графе»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Игнатюк Д.А.

Проверил:
Козинский А.А.

Лабораторная работа №3

«Нахождение кратчайшего пути в графе»

Цель работы: освоить алгоритмы Дейкстры и Флойда–Уоршелла для нахождения кратчайших путей в взвешенном графе и реализовать их на языке C.

Ход работы

Код

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#define INF 1000000000000000000LL // «бесконечность»
// ПРЕДСТАВЛЕНИЕ ГРАФА: СПИСКИ СМЕЖНОСТИ (со весами)
typedef struct Edge {
    int v; // соседняя вершина
    long long w; // вес ребра
    struct Edge* next;
} Edge;
typedef struct {
    int n;
    int m;
    Edge** adj;
} Graph;
// Добавление неориентированного взвешенного ребра
/* direction = 1 – только u→v (направленное)
   direction = 0 – и u→v, и v→u (неориентированное) */
static void add_edge(Graph* g, int u, int v, long long w, int direction)
{
    Edge* p = (Edge*)malloc(sizeof(Edge));
    p->v = v; p->w = w; p->next = g->adj[u]; g->adj[u] = p;
    if (!direction) {
        p = (Edge*)malloc(sizeof(Edge));
        p->v = u; p->w = w; p->next = g->adj[v]; g->adj[v] = p;
    }
    g->m++;
}
// Чтение графа из файла
// Формат: первая строка – n; далее строки u v w
static int read_graph(const char* fname, Graph* g)
{
    FILE* f = fopen(fname, "r");
    if (!f) { fprintf(stderr, "не открыт %s\n", fname); return 0; }
    char line[256];
    int n = 0;
    int header_ok = 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &n) == 1) { header_ok = 1; break; }
    }
    if (!header_ok || n < 0) {
        fprintf(stderr, "%s: плохой формат\n", fname);
        fclose(f); return 0;
    }
    g->n = n;
    g->m = 0;
    g->adj = (Edge**)calloc(n + 1, sizeof(Edge));
    if (!g->adj) { fclose(f); return 0; }
```

```

while (fgets(line, sizeof(line), f)) {
    int u = 0, v = 0;
    long long w = 0;
    int got = sscanf(line, "%d %d %lld", &u, &v, &w);
    if (got < 3) continue; // нужны все три числа
    if (u < 1 || u > g->n || v < 1 || v > g->n) {
        fprintf(stderr, "%s: rebro (%d,%d) vne [1..%d], propusk\n",
            fname, u, v, g->n);
        continue;
    }
    if (u == v) {
        fprintf(stderr, "%s: petlya (%d,%d), propusk\n", fname, u,
v);
        continue;
    }
    // Отрицательные веса ДОПУСКАЮТСЯ – их обрабатывает Флойд.
    // Дейкстра будет пропущена (см. main).
    int direction = (w < 0) ? 1 : 0; // отриц. вес → направленное
    add_edge(g, u, v, w, direction);
}
fclose(f);
return 1;
}
// Освобождение
static void free_graph(Graph* g)
{
    for (int i = 1; i <= g->n; i++) {
        Edge* e = g->adj[i];
        while (e) { Edge* nx = e->next; free(e); e = nx; }
    }
    free(g->adj);
    g->adj = NULL;
    g->n = g->m = 0;
}
/* ПРОВЕРКА: есть ли в графе ребро с отрицательным весом.
Возвращает 1, если хотя бы одно ребро отрицательно. */
static int has_negative_edges(const Graph* g)
{
    for (int i = 1; i <= g->n; i++)
        for (const Edge* e = g->adj[i]; e; e = e->next)
            if (e->w < 0) return 1;
    return 0;
}
/* АЛГОРИТМ ДЕЙКСТРЫ
Находит кратчайшие расстояния от стартовой вершины s до всех.
dist[i] – расстояние от s до i
prev[i] – предшественник i на кратчайшем пути
visited[i] – 1, если вершина уже «закрыта» */
static void dijkstra(const Graph* g, int s, long long* dist, int* prev)
{
    int N = g->n;
    int* visited = (int*)calloc(N + 1, sizeof(int));
    for (int i = 1; i <= N; i++) {
        dist[i] = INF;
        prev[i] = 0;
    }
    dist[s] = 0;
    for (int step = 0; step < N; step++) {
        // Выбираем непосещённую вершину с минимальным dist

```

```

    int u = -1;
    long long best = INF;
    for (int i = 1; i <= N; i++) {
        if (!visited[i] && dist[i] < best) {
            best = dist[i];
            u = i;
        }
    }
    if (u == -1) break;    // все оставшиеся недостижимы

    visited[u] = 1;
    // Релаксация соседей
    for (const Edge* e = g->adj[u]; e; e = e->next) {
        int v = e->v;
        if (!visited[v] && dist[u] + e->w < dist[v]) {
            dist[v] = dist[u] + e->w;
            prev[v] = u;
        }
    }
}
free(visited);
}
// Восстановление пути s → t по массиву prev
static void print_path(int* prev, int s, int t)
{
    if (t == s) { printf("%d", s); return; }
    if (prev[t] == 0) { printf("(нет пути)"); return; }
    // Рекурсивно не надо — идём назад и разворачиваем
    int stack[1024];
    int top = 0;
    int cur = t;
    while (cur != 0) {
        stack[top++] = cur;
        if (cur == s) break;
        cur = prev[cur];
    }
    for (int i = top - 1; i >= 0; i--) {
        printf("%d", stack[i]);
        if (i > 0) printf(" -> ");
    }
}
/* АЛГОРИТМ ФЛОЙДА-УОРШЕЛЛА
d[i][j] — кратчайшее расстояние между i и j.
next[i][j] — следующая вершина на пути i → j (для восстановления). */
static void floyd_warshall(const Graph* g, long long** d, int** next)
{
    int N = g->n;
    // Инициализация
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;
            next[i][j] = 0;
        }
    }
    // Прямые рёбра
    for (int u = 1; u <= N; u++) {
        for (const Edge* e = g->adj[u]; e; e = e->next) {
            int v = e->v;

```

```

        if (e->w < d[u][v]) {
            d[u][v] = e->w;
            next[u][v] = v;
        }
    }
}
// Тройной цикл
for (int k = 1; k <= N; k++) {
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            if (d[i][k] < INF && d[k][j] < INF) {
                long long through = d[i][k] + d[k][j];
                if (through < d[i][j]) {
                    d[i][j] = through;
                    next[i][j] = next[i][k];
                }
            }
        }
    }
}
}
/* ПРОВЕРКА НА ОТРИЦАТЕЛЬНЫЙ ЦИКЛ.
   После работы Флойда, если на диагонали есть d[i][i] < 0,
   то через вершину i проходит отрицательный цикл.
   Возвращает 1, если цикл найден. */
static int has_negative_cycle(long long** d, int N)
{
    for (int i = 1; i <= N; i++)
        if (d[i][i] < 0) return 1;
    return 0;
}
int main(int argc, char** argv)
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <graph-file>\n", argv[0]);
        return 1;
    }
    Graph g;
    if (!read_graph(argv[1], &g)) return 1;
    printf("Fayl: %s\n", argv[1]);
    printf("Vershin: %d, ryober: %d\n\n", g.n, g.m);
    int N = g.n;
    // АЛГОРИТМ ДЕЙКСТРЫ от вершины 1 — только если нет отрицательных
    ребер
    printf("=== ALGORITM DEYKSTRY (ot vershiny 1) ===\n");
    if (has_negative_edges(&g)) {
        printf("V grafe est otricatelnye rebra.\n");
        printf("Algoritm Deykstry ne primenim — propuskaem.\n\n");
    }
    else {
        long long* dist = (long long*)malloc((N + 1) * sizeof(long
long));
        int* prev = (int*)malloc((N + 1) * sizeof(int));
        if (!dist || !prev) {
            free(dist); free(prev); free_graph(&g); return 1;
        }
        dijkstra(&g, 1, dist, prev);

        printf("Krachaishie rasstoyaniya ot 1:\n");

```

```

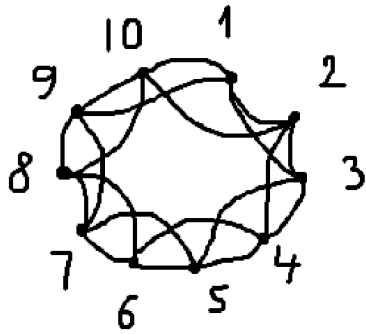
        for (int i = 1; i <= N; i++) {
            printf(" do %2d: ", i);
            if (dist[i] >= INF) printf("nedostizhima\n");
            else printf("%lld\n", dist[i]);
        }
        printf("\nKrachaishie puti ot 1:\n");
        for (int i = 1; i <= N; i++) {
            if (i == 1) continue;
            printf(" do %2d: ", i);
            if (dist[i] >= INF) printf("(net puti)\n");
            else { print_path(prev, 1, i); printf(" (dlina %lld)\n",
dist[i]); }
        }
        free(dist);
        free(prev);
        printf("\n");
    }
    // АЛГОРИТМ ФЛОЙДА-УОРШЕЛЛА — работает всегда
    printf("=== ALGORITM FLOYD-WARSHALL ===\n");
    long long** d = (long long**)malloc((N + 1) * sizeof(long long*));
    int** nx = (int**)malloc((N + 1) * sizeof(int*));
    for (int i = 0; i <= N; i++) {
        d[i] = (long long*)malloc((N + 1) * sizeof(long long));
        nx[i] = (int*)malloc((N + 1) * sizeof(int));
    }
    floyd_warshall(&g, d, nx);
    // Проверка на отрицательный цикл
    if (has_negative_cycle(d, N)) {
        printf("V grafe obnaruzhen otricatelnyy cikl!\n");
        printf("Krachaishie rasstoyaniya ne opredeleny (stremyatsya k -
inf).\n");
    }
    else {
        printf("Matritsa krachaishih rasstoyaniy:\n ");
        for (int j = 1; j <= N; j++) printf("%15d", j);
        printf("\n");
        for (int i = 1; i <= N; i++) {
            printf("%4d:", i);
            for (int j = 1; j <= N; j++) {
                if (d[i][j] >= INF) printf("%15s", "INF");
                else printf("%15lld", d[i][j]);
            }
            printf("\n");
        }
        // Кратчайшие пути от 1 до всех по Флойду
        printf("\nKrachaishie rasstoyaniya ot 1 (Floyd):\n");
        for (int j = 1; j <= N; j++) {
            printf(" do %2d: ", j);
            if (d[1][j] >= INF) printf("nedostizhima\n");
            else printf("%lld\n", d[1][j]);
        }
    }
    for (int i = 0; i <= N; i++) { free(d[i]); free(nx[i]); }
    free(d);
    free(nx);
    free_graph(&g);
    return 0;
}

```

Проверки:

Тест01 - cirkulyant_10vershin

```
10
1 2 1536819634
1 3 -705715377972
2 3 -30
2 4 -673
3 4 85
3 5 0
4 5 -4274382698
4 6 -216
5 6 -2
5 7 5290970
6 7 1000000000000
6 8 -96334155009
7 8 -691376
7 9 38
8 9 -66159
8 10 -41452
9 1 -61535661690
9 10 34815457
10 1 -231
10 2 -5011120308
```



Fayl: tests/test01.txt

Vershin: 10, ryober: 20

=== ALGORITM DEYKSTRY (ot vershiny 1) ===

V grafe est otricateľnye rebra.

Algoritm Deykstry ne primenim 4 propuskaem.

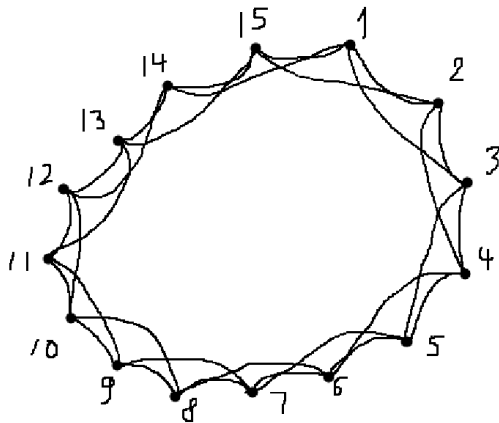
=== ALGORITM FLOYDWARSHALL ===

V grafe obnaruzhen otricateľnyy cikl!

Krachaishie rasstoyaniya ne opredeleny (stremyatsya k -inf).

Тест02 - cirkulyant_15vershin

```
15
1 2 20
1 3 425165329989
2 3 37890
2 4 701
3 4 974754451
3 5 3453222462
4 5 10096
4 6 477641
5 6 735044390115
5 7 744
6 7 28676693
6 8 98
7 8 68536266820
7 9 6417963406
8 9 541
8 10 92
9 10 290341638342
9 11 95441357
10 11 173017
10 12 657876900
11 12 9904
11 13 4757753
12 13 98613689275
12 14 73596179260
13 14 65
13 15 1000000000000
14 1 227654057789
14 15 605966213
15 1 10632555939
15 2 672867
```



Fayl: tests/test02.txt

Vershin: 15, ryober: 30

=== ALGORITM DEYKSTRY (ot vershiny 1) ===

Krachaishie rasstoyaniya ot 1:

```
do 1: 0
do 2: 20
do 3: 37910
do 4: 721
do 5: 10817
do 6: 478362
do 7: 11561
do 8: 478460
do 9: 479001
do 10: 478552
do 11: 651569
do 12: 661473
do 13: 5409322
do 14: 5409387
do 15: 672887
```

Krachaishie puti ot 1:

```
do 2: 1 -> 2 (dlina 20)
do 3: 1 -> 2 -> 3 (dlina 37910)
do 4: 1 -> 2 -> 4 (dlina 721)
do 5: 1 -> 2 -> 4 -> 5 (dlina 10817)
do 6: 1 -> 2 -> 4 -> 6 (dlina 478362)
do 7: 1 -> 2 -> 4 -> 5 -> 7 (dlina 11561)
do 8: 1 -> 2 -> 4 -> 6 -> 8 (dlina 478460)
do 9: 1 -> 2 -> 4 -> 6 -> 8 -> 9 (dlina 479001)
do 10: 1 -> 2 -> 4 -> 6 -> 8 -> 10 (dlina 478552)
do 11: 1 -> 2 -> 4 -> 6 -> 8 -> 10 -> 11 (dlina 651569)
do 12: 1 -> 2 -> 4 -> 6 -> 8 -> 10 -> 11 -> 12 (dlina 661473)
do 13: 1 -> 2 -> 4 -> 6 -> 8 -> 10 -> 11 -> 13 (dlina 5409322)
do 14: 1 -> 2 -> 4 -> 6 -> 8 -> 10 -> 11 -> 13 -> 14 (dlina 5409387)
do 15: 1 -> 2 -> 15 (dlina 672887)
```

```

=== ALGORITM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:

```

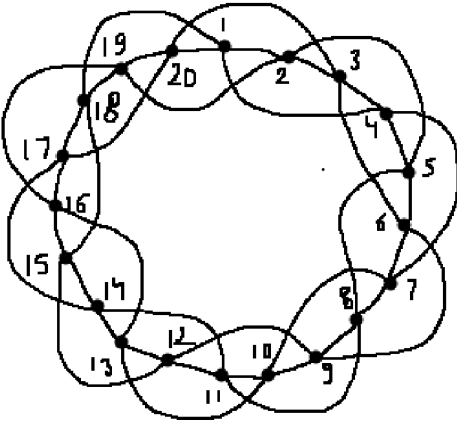
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1:	0	20	37910	721	10817	478362	11561	478460	479081	478552	651569	661473	5409322	5409367	672887
2:	20	0	37890	701	10797	478342	11541	478440	478981	478532	651549	661453	5409302	5409367	672867
3:	37910	37890	0	38591	48687	516232	40431	516330	516871	516422	689439	699343	5447192	5447257	710757
4:	721	701	38591	0	10096	477641	10040	477739	478280	477831	658848	660752	5408661	5408666	673568
5:	10817	10797	48687	10096	0	487737	744	487635	488376	487927	660944	670848	5418697	5418762	683664
6:	478362	478342	516232	477641	487737	0	488481	98	639	190	173207	183111	4930960	4931025	1151209
7:	11561	11541	40431	10040	744	488481	0	488579	489120	488671	661688	671592	5419441	5419506	684408
8:	478460	478440	516330	477739	487635	98	488579	0	541	92	172109	183013	4930862	4930927	1151307
9:	479081	478981	516871	478280	488376	639	489120	541	0	633	173650	183554	4931403	4931468	1151848
10:	478552	478532	516422	477831	487927	190	488671	92	633	0	173017	182921	4930770	4930835	1151399
11:	651569	651549	689439	658848	660944	173207	661688	172109	173650	173017	0	9904	4757753	4757818	1224416
12:	661473	661453	699343	660752	670848	183111	671592	183013	183554	182921	9904	0	4767657	4767722	1334320
13:	5409322	5409302	5447192	5408661	5418697	4930960	5419441	4930862	4931403	4930770	4757753	4767657	0	65	6882169
14:	5409367	5409367	5447257	5408666	5418762	4930927	5419506	4930835	4931468	4930770	4757818	4767722	65	0	6882234
15:	672887	672867	710757	673568	683664	1151209	684408	1151307	1151848	1151399	1224416	1334320	6882169	6882234	0

```

Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 20
do 3: 37910
do 4: 721
do 5: 10817
do 6: 478362
do 7: 11561
do 8: 478460
do 9: 479081
do 10: 478552
do 11: 651569
do 12: 661473
do 13: 5409322
do 14: 5409367
do 15: 672887

```

Тест03 - cirkulyant_20vershin



```

20
1 2 1000000000000
1 4 1000000000000
2 3 1000000000000
2 5 1000000000000
3 4 1000000000000
3 6 1000000000000
4 5 1000000000000
4 7 1000000000000
5 6 1000000000000
5 8 1000000000000
6 7 1000000000000
6 9 1000000000000
7 8 1000000000000
7 10 1000000000000
8 9 1000000000000
8 11 1000000000000
9 10 1000000000000
9 12 1000000000000
10 11 1000000000000
10 13 1000000000000
11 12 1000000000000

```

```

Fayl: tests/test03.txt
Vershin: 20, ryober: 40

=== ALGORITM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 1000000000000
do 3: 2000000000000
do 4: 1000000000000
do 5: 2000000000000
do 6: 3000000000000
do 7: 2000000000000
do 8: 3000000000000
do 9: 4000000000000
do 10: 3000000000000
do 11: 4000000000000
do 12: 3000000000000
do 13: 4000000000000
do 14: 3000000000000
do 15: 2000000000000
do 16: 3000000000000
do 17: 2000000000000
do 18: 1000000000000
do 19: 2000000000000
do 20: 1000000000000

Krachaishie puti ot 1:
do 2: 1 -> 2 (dlina 1000000000000)
do 3: 1 -> 2 -> 3 (dlina 2000000000000)
do 4: 1 -> 4 (dlina 1000000000000)
do 5: 1 -> 2 -> 5 (dlina 2000000000000)
do 6: 1 -> 2 -> 3 -> 6 (dlina 3000000000000)
do 7: 1 -> 4 -> 7 (dlina 2000000000000)
do 8: 1 -> 2 -> 5 -> 8 (dlina 3000000000000)
do 9: 1 -> 2 -> 3 -> 6 -> 9 (dlina 4000000000000)
do 10: 1 -> 4 -> 7 -> 10 (dlina 3000000000000)
do 11: 1 -> 2 -> 5 -> 8 -> 11 (dlina 4000000000000)
do 12: 1 -> 18 -> 15 -> 12 (dlina 3000000000000)
do 13: 1 -> 4 -> 7 -> 10 -> 13 (dlina 4000000000000)
do 14: 1 -> 18 -> 15 -> 14 (dlina 3000000000000)
do 15: 1 -> 18 -> 15 (dlina 2000000000000)
do 16: 1 -> 18 -> 15 -> 16 (dlina 3000000000000)
do 17: 1 -> 18 -> 17 (dlina 2000000000000)
do 18: 1 -> 18 (dlina 1000000000000)
do 19: 1 -> 2 -> 19 (dlina 2000000000000)
do 20: 1 -> 20 (dlina 1000000000000)

```

```

Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 1000000000000
do 3: 2000000000000
do 4: 1000000000000
do 5: 2000000000000
do 6: 3000000000000
do 7: 2000000000000
do 8: 3000000000000
do 9: 4000000000000
do 10: 3000000000000
do 11: 4000000000000
do 12: 3000000000000
do 13: 4000000000000
do 14: 3000000000000
do 15: 2000000000000
do 16: 3000000000000
do 17: 2000000000000
do 18: 1000000000000
do 19: 2000000000000
do 20: 1000000000000

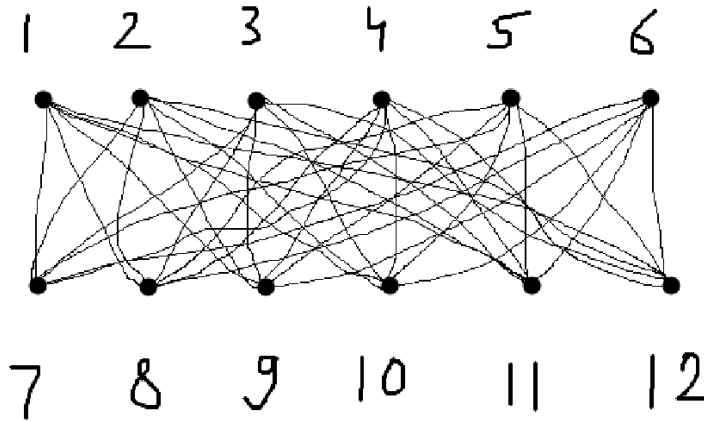
```

```

=== ALGORITHM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:
1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
2: 1000000000 0 1000000000 2000000000 1000000000 2000000000 3000000000 2000000000 4000000000 3000000000 4000000000 3000000000 4000000000 3000000000 2000000000 3000000000 2000000000 1000000000 2000000000 1000000000
3: 1000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
4: 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000
5: 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
6: 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000
7: 3000000000 2000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
8: 2000000000 1000000000 2000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000
9: 4000000000 3000000000 2000000000 2000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
10: 3000000000 2000000000 1000000000 2000000000 2000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000
11: 4000000000 3000000000 4000000000 3000000000 2000000000 2000000000 1000000000 2000000000 1000000000 2000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
12: 3000000000 2000000000 4000000000 3000000000 2000000000 2000000000 2000000000 1000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000
13: 4000000000 3000000000 4000000000 3000000000 2000000000 2000000000 2000000000 2000000000 1000000000 2000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000
14: 3000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000 2000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000 1000000000
15: 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 2000000000 1000000000
16: 3000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000 1000000000
17: 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 2000000000 1000000000
18: 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000 1000000000
19: 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0 1000000000
20: 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 2000000000 1000000000 0

```

Тест04 - K66_12vershin



```

12
1 7 -1
1 8 2
1 9 3
1 10 4
1 11 5
1 12 6
2 7 7
2 8 8
2 9 9
2 10 10
2 11 11
2 12 12
3 7 13

```

```

Fayl: tests/test04.txt
Vershin: 12, ryober: 36

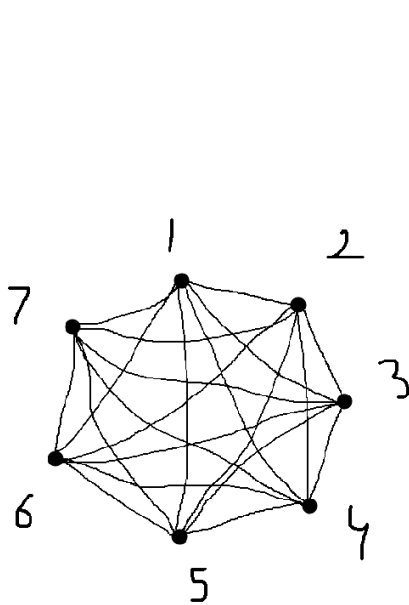
=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
V grafe est otricateelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.

=== ALGORITHM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:
1 2 3 4 5 6 7 8 9 10 11 12
1: 0 6 12 18 24 30 -1 2 3 4 5 6
2: 10 0 20 26 32 38 7 8 9 10 11 12
3: 16 20 0 32 38 44 13 14 15 16 17 18
4: 22 26 32 0 44 50 19 20 21 22 23 24
5: 28 32 38 44 0 56 25 26 27 28 29 30
6: 34 38 44 50 56 0 31 32 33 34 35 36
7: 17 7 13 19 25 31 0 15 16 17 18 19
8: 2 8 14 20 26 32 1 0 5 6 7 8
9: 3 9 15 21 27 33 2 5 0 7 8 9
10: 4 10 16 22 28 34 3 6 7 0 9 10
11: 5 11 17 23 29 35 4 7 8 9 0 11
12: 6 12 18 24 30 36 5 8 9 10 11 0

Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 6
do 3: 12
do 4: 18
do 5: 24
do 6: 30
do 7: -1
do 8: 2
do 9: 3
do 10: 4
do 11: 5
do 12: 6

```

Тест05 - K7_7vershin



```

7
1 2 -1
1 3 -2
1 4 -3
1 5 -4
1 6 -5
1 7 -6
2 3 1
2 4 1
2 5 1
2 6 1
2 7 1
3 4 1
3 5 1
3 6 1
3 7 1
4 5 1
4 6 1
4 7 1
5 6 1
5 7 1
6 7 1

```

```

Fayl: tests/test05.txt
Vershin: 7, ryober: 21

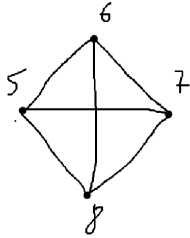
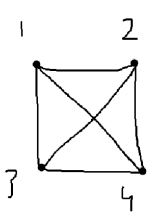
=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
V grafe est otricateelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.

=== ALGORITHM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:
1 2 3 4 5 6 7
1: 0 -5 -5 -5 -5 -5 -6
2: INF 0 1 1 1 1 1
3: INF 1 0 1 1 1 1
4: INF 1 1 0 1 1 1
5: INF 1 1 1 0 1 1
6: INF 1 1 1 1 0 1
7: INF 1 1 1 1 1 0

Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: -5
do 3: -5
do 4: -5
do 5: -5
do 6: -5
do 7: -6

```

Тест06 - eyler06_dva_K4_8vershin



```

8
1 2 3
1 3 4
1 4 5
2 3 5
2 4 6
3 4 7
5 6 3
5 7 4
5 8 5
6 7 5
6 8 6
7 8 7

```

Fayl: tests/test06.txt
Vershin: 8, ryober: 12

```

=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 3
do 3: 4
do 4: 5
do 5: nedostizhima
do 6: nedostizhima
do 7: nedostizhima
do 8: nedostizhima

```

Krachaishie puti ot 1:

```

do 2: 1 -> 2 (dlina 3)
do 3: 1 -> 3 (dlina 4)
do 4: 1 -> 4 (dlina 5)
do 5: (net puti)
do 6: (net puti)
do 7: (net puti)
do 8: (net puti)

```

=== ALGORITHM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:

	1	2	3	4	5	6	7	8
1:	0	3	4	5	INF	INF	INF	INF
2:	3	0	5	6	INF	INF	INF	INF
3:	4	5	0	7	INF	INF	INF	INF
4:	5	6	7	0	INF	INF	INF	INF
5:	INF	INF	INF	INF	0	3	4	5
6:	INF	INF	INF	INF	3	0	5	6
7:	INF	INF	INF	INF	4	5	0	7
8:	INF	INF	INF	INF	5	6	7	0

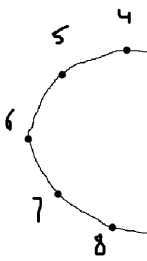
Krachaishie rasstoyaniya ot 1 (Floyd):

```

do 1: 0
do 2: 3
do 3: 4
do 4: 5
do 5: nedostizhima
do 6: nedostizhima
do 7: nedostizhima
do 8: nedostizhima

```

Тест07 - pesochnye_chasy_19vershin



```

19
1 2 -1
2 3 -1
3 4 -1
4 5 -1
5 6 -1
6 7 -1
7 8 -1
8 9 -1
9 10 -1
10 1 10
11 10 0
11 12 -1
12 13 -1
13 14 -1
14 15 -1
15 16 -1
16 17 -1
17 18 -1
18 19 -1
19 1 -1

```

Fayl: tests/test07.txt
Vershin: 19, ryober: 20

=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
V grafe est otricatelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.

=== ALGORITHM FLOYDWARSHALL ===
Matritsa krachaishih rasstoyaniy:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1:	0	-1	-2	-3	-4	-5	-6	-7	-8	-9	10	9	8	7	6	5	4	3	2
2:	2	0	-1	-2	-3	-4	-5	-6	-7	-8	12	11	10	9	8	7	6	5	4
3:	3	2	0	-1	-2	-3	-4	-5	-6	-7	13	12	11	10	9	8	7	6	5
4:	4	3	2	0	-1	-2	-3	-4	-5	-6	14	13	12	11	10	9	8	7	6
5:	5	4	3	2	0	-1	-2	-3	-4	-5	15	14	13	12	11	10	9	8	7
6:	6	5	4	3	2	0	-1	-2	-3	-4	16	15	14	13	12	11	10	9	8
7:	7	6	5	4	3	2	0	-1	-2	-3	17	16	15	14	13	12	11	10	9
8:	8	7	6	5	4	3	2	0	-1	-2	18	17	16	15	14	13	12	11	10
9:	9	8	7	6	5	4	3	2	0	-1	19	18	17	16	15	14	13	12	11
10:	10	9	8	7	6	5	4	3	2	0	20	19	18	17	16	15	14	13	12
11:	-9	-10	-11	-12	-13	-14	-15	-16	-17	-18	0	-1	-2	-3	-4	-5	-6	-7	-8
12:	-8	-9	-10	-11	-12	-13	-14	-15	-16	-17	2	0	-1	-2	-3	-4	-5	-6	-7
13:	-7	-8	-9	-10	-11	-12	-13	-14	-15	-16	3	2	0	-1	-2	-3	-4	-5	-6
14:	-6	-7	-8	-9	-10	-11	-12	-13	-14	-15	4	3	2	0	-1	-2	-3	-4	-5
15:	-5	-6	-7	-8	-9	-10	-11	-12	-13	-14	5	4	3	2	0	-1	-2	-3	-4
16:	-4	-5	-6	-7	-8	-9	-10	-11	-12	-13	6	5	4	3	2	0	-1	-2	-3
17:	-3	-4	-5	-6	-7	-8	-9	-10	-11	-12	7	6	5	4	3	2	0	-1	-2
18:	-2	-3	-4	-5	-6	-7	-8	-9	-10	-11	8	7	6	5	4	3	2	0	-1
19:	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10	9	8	7	6	5	4	3	2	0

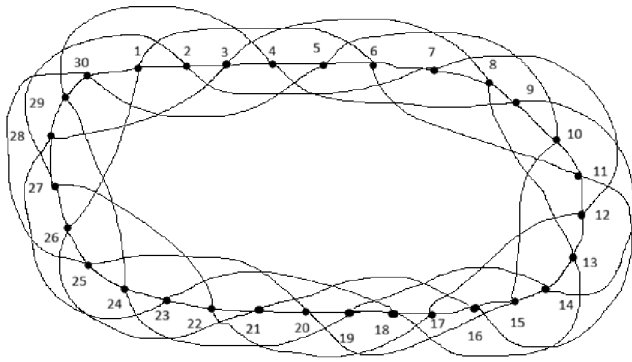
Krachaishie rasstoyaniya ot 1 (Floyd):

```

do 1: 0
do 2: -1
do 3: -2
do 4: -3
do 5: -4
do 6: -5
do 7: -6
do 8: -7
do 9: -8
do 10: -9
do 11: 10
do 12: 9
do 13: 8
do 14: 7
do 15: 6
do 16: 5
do 17: 4
do 18: 3
do 19: 2

```

Tect08 – cirkulyant_30vershin



```
Fayl: tests/test08.txt
Vershin: 30, ryober: 60
```

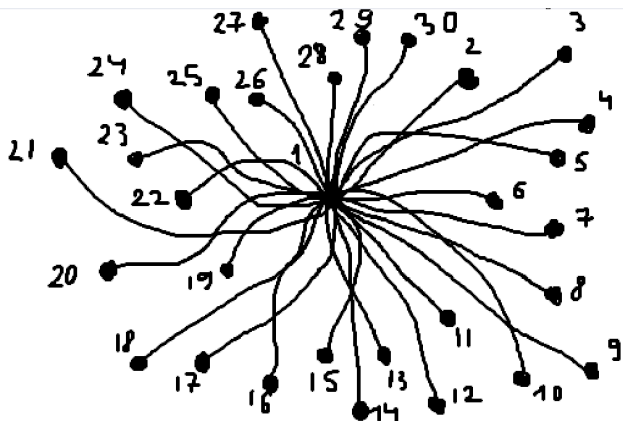
```
=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 1000
do 3: 2000
do 4: 3000
```

```
Krachaishie puti ot 1:
do 2: 1 -> 2      (dlina 1000)
do 3: 1 -> 2 -> 3  (dlina 2000)
do 4: 1 -> 2 -> 3 -> 4 (dlina 3000)
do 5: 1 -> 6 -> 5  (dlina 2000)
do 6: 1 -> 6      (dlina 1000)
do 7: 1 -> 2 -> 7  (dlina 2000)
do 8: 1 -> 2 -> 3 -> 8 (dlina 3000)
do 9: 1 -> 2 -> 3 -> 4 -> 9 (dlina 4000)
do 10: 1 -> 6 -> 5 -> 10 (dlina 3000)
do 11: 1 -> 6 -> 11 (dlina 2000)
do 12: 1 -> 2 -> 7 -> 12 (dlina 3000)
do 13: 1 -> 2 -> 3 -> 8 -> 13 (dlina 4000)
do 14: 1 -> 2 -> 3 -> 4 -> 9 -> 14 (dlina 5000)
do 15: 1 -> 6 -> 5 -> 10 -> 15 (dlina 4000)
do 16: 1 -> 6 -> 11 -> 16 (dlina 3000)
do 17: 1 -> 2 -> 7 -> 12 -> 17 (dlina 4000)
do 18: 1 -> 2 -> 3 -> 8 -> 13 -> 18 (dlina 5000)
do 19: 1 -> 26 -> 21 -> 20 -> 19 (dlina 4000)
do 20: 1 -> 26 -> 21 -> 20 (dlina 3000)
do 21: 1 -> 26 -> 21 (dlina 2000)
do 22: 1 -> 26 -> 21 -> 22 (dlina 3000)
do 23: 1 -> 26 -> 21 -> 22 -> 23 (dlina 4000)
do 24: 1 -> 26 -> 25 -> 24 (dlina 3000)
do 25: 1 -> 26 -> 25 (dlina 2000)
do 26: 1 -> 26 (dlina 1000)
do 27: 1 -> 2 -> 27 (dlina 2000)
do 28: 1 -> 2 -> 3 -> 28 (dlina 3000)
do 29: 1 -> 30 -> 29 (dlina 2000)
do 30: 1 -> 30 (dlina 1000)
```

```
Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 1000
do 3: 2000
do 4: 3000
do 5: 2000
do 6: 1000
do 7: 2000
do 8: 3000
do 9: 4000
do 10: 3000
do 11: 2000
do 12: 3000
do 13: 4000
do 14: 5000
do 15: 4000
do 16: 3000
do 17: 4000
do 18: 5000
do 19: 4000
do 20: 3000
do 21: 2000
do 22: 3000
do 23: 4000
do 24: 3000
do 25: 2000
do 26: 1000
do 27: 2000
do 28: 3000
do 29: 2000
do 30: 1000
```

=== ALGORITHM FLOYDWARSHALL ===																														
Matritsa kraichaishtin rassvoyoizni:																														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
1:	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000	1000	2000	3000	2000	1000
2:	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000	1000	2000	3000	2000
3:	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000	1000	2000	3000
4:	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000	1000	2000
5:	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000	1000
6:	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000	2000
7:	2000	1000	3000	2000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000	3000
8:	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000	4000
9:	4000	3000	2000	3000	2000	1000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000	3000
10:	3000	4000	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000	2000
11:	2000	3000	4000	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000	3000
12:	3000	2000	3000	4000	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000	4000
13:	4000	3000	2000	3000	4000	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000	4000	5000	4000	3000	4000	5000
14:	5000	4000	3000	2000	3000	4000	3000	2000	1000	2000	3000	2000	1000	0	1000	2000	3000	2000	1000	2000	3000	4000	3000	2000	3000					

Tect09 – zvezda 30vershin



```

=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
V grafe est otricatelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.

```

[illegible]

do 1: 0
do 2: 59453716
do 3: -45
do 4: 823
do 5: 38900975
do 6: 1152
do 7: 904
do 8: -736867156
do 9: -95530
do 10: 2
do 11: 5
do 12: -54
do 13: -6655
do 14: 9969337655
do 15: 419515662813
do 16: -40218146238
do 17: 713
do 18: 852978062
do 19: 740318
do 20: -1490370
do 21: 39191322174
do 22: 9
do 23: 415558381
do 24: 9051739
do 25: 65811281236
do 26: 56304762
do 27: 4183409495
do 28: 30932
do 29: 350151790144
do 30: -7

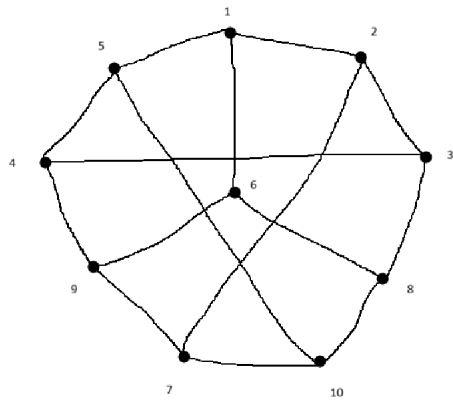
```

=== ALGORITM DEYKSTRY (ot vershiny 1) ===
V grafe est otricatelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.

```

```
do 1: 0
do 2: 985645255
do 3: 605144438
do 4: 989885381
do 5: 989845255
do 6: 672393541
do 7: 846373554173
do 8: 989884998
do 9: 989894899
do 10: 985845705
do 11: 985853805
do 12: 672404271
do 13: -64448613978
do 14: 847744993805
do 15: 84963388168
do 16: 989146165
do 17: 1077754151
do 18: 134385421
do 19: 298599953676
do 20: 992891613
do 21: 984672756
do 22: 963289735
do 23: 93994931882
do 24: 672495720
do 25: 672404805
do 26: -6424085286
do 27: -6445963758
do 28: 84710508776
do 29: 84744091999
do 30: 84968914866
do 31: 84963388631
```

Тест11 – Petersen_10vershin



```

10
1 2 43170240988
2 3 74
3 4 48
4 5 424159022
5 1 6292047
6 8 2475
8 10 53173080415
10 7 27767652
7 9 4591549
9 6 840674872437
1 6 79
2 7 574
3 8 78917692
4 9 91924781384
5 10 363709366|
    
```

Fayl: tests/test11.txt
Vershin: 10, ryober: 15

=== ALGORITHM DEJKSTRY (ot vershiny 1) ===

Krachaishie rasstoyaniya ot 1:

```

do 1: 0
do 2: 78920320
do 3: 78920246
do 4: 78920294
do 5: 6292047
do 6: 79
do 7: 78920894
do 8: 2554
do 9: 83512443
do 10: 106688546
    
```

Krachaishie puti ot 1:

```

do 2: 1 -> 6 -> 8 -> 3 -> 2 (dlina 78920320)
do 3: 1 -> 6 -> 8 -> 3 (dlina 78920246)
do 4: 1 -> 6 -> 8 -> 3 -> 4 (dlina 78920294)
do 5: 1 -> 5 (dlina 6292047)
do 6: 1 -> 6 (dlina 79)
do 7: 1 -> 6 -> 8 -> 3 -> 2 -> 7 (dlina 78920894)
do 8: 1 -> 6 -> 8 (dlina 2554)
do 9: 1 -> 6 -> 8 -> 3 -> 2 -> 7 -> 9 (dlina 83512443)
do 10: 1 -> 6 -> 8 -> 3 -> 2 -> 7 -> 10 (dlina 106688546)
    
```

Krachaishie rasstoyaniya ot 1 (Floyd):

```

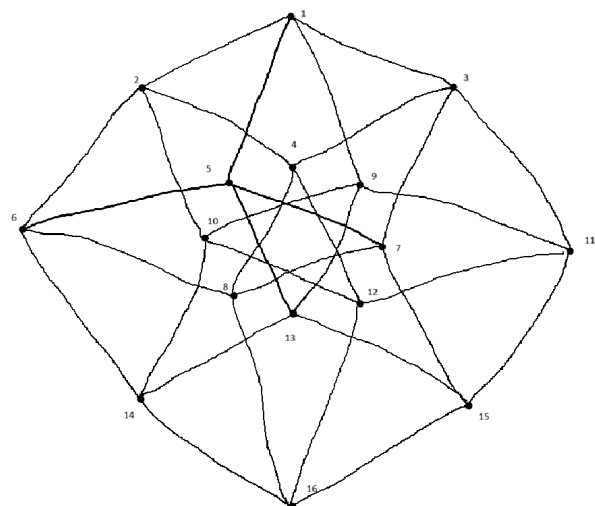
do 1: 0
do 2: 78920320
do 3: 78920246
do 4: 78920294
do 5: 6292047
do 6: 79
do 7: 78920894
do 8: 2554
do 9: 83512443
do 10: 106688546
    
```

=== ALGORITHM FLOYDWARSHALL ===

Matritsa krachaishih rasstoyaniy:

	1	2	3	4	5	6	7	8	9	10
1:	0	78920320	78920246	78920294	6292047	79	78920894	2554	83512443	106688546
2:	78920320	0	74	122	85212367	78920241	574	78917766	4592123	27768226
3:	78920246	74	0	48	85212293	78920167	648	78917692	4592197	27768300
4:	78920294	122	48	0	85212341	78920215	696	78917740	4592245	27768348
5:	6292047	85212367	85212293	85212341	0	6292126	85212941	6294601	89804490	112980593
6:	79	78920241	78920167	78920215	6292126	0	78920815	2475	83512364	106688467
7:	78920894	574	648	696	85212941	78920815	0	78918340	4591549	27767652
8:	2554	78917766	78917692	78917740	6294601	2475	78918340	0	83509889	106685992
9:	83512443	4592123	4592197	4592245	89804490	83512364	4591549	83509889	0	32359201
10:	106688546	27768226	27768300	27768348	112980593	106688467	27767652	106685992	32359201	0

Тест12 – giperkub_Q4_16vershin



```

16
1 2 2
1 3 3
1 5 5
1 9 9
2 4 8
2 6 12
2 10 20
3 4 12
3 7 21
3 11 33
4 8 32
4 12 48
5 6 30
5 7 35
5 13 65
6 8 48
6 14 84
7 8 56
7 15 105
8 16 128
9 10 90
9 11 99
9 13 117
10 12 120
10 14 140
11 12 132
11 15 165
12 16 184
13 14 182
13 15 195
14 16 224
15 16 240|
    
```

```
Fayl: tests/test12.txt
Vershin: 16, ryober: 32

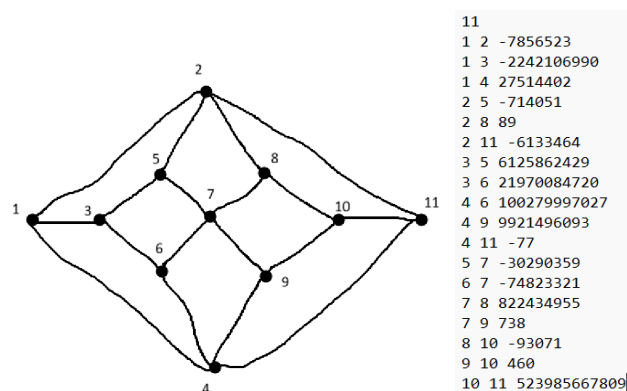
=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 2
do 3: 3
do 4: 10
do 5: 5
do 6: 14
do 7: 24
do 8: 42
do 9: 9
do 10: 22
do 11: 36
do 12: 58
do 13: 70
do 14: 98
do 15: 129
do 16: 170
```

```
Krachaishie puti ot 1:
do 2: 1 -> 2 (dlina 2)
do 3: 1 -> 3 (dlina 3)
do 4: 1 -> 2 -> 4 (dlina 10)
do 5: 1 -> 5 (dlina 5)
do 6: 1 -> 2 -> 6 (dlina 14)
do 7: 1 -> 3 -> 7 (dlina 24)
do 8: 1 -> 2 -> 4 -> 8 (dlina 42)
do 9: 1 -> 9 (dlina 9)
do 10: 1 -> 2 -> 10 (dlina 22)
do 11: 1 -> 3 -> 11 (dlina 36)
do 12: 1 -> 2 -> 4 -> 12 (dlina 58)
do 13: 1 -> 5 -> 13 (dlina 70)
do 14: 1 -> 2 -> 6 -> 14 (dlina 98)
do 15: 1 -> 3 -> 7 -> 15 (dlina 129)
do 16: 1 -> 2 -> 4 -> 8 -> 16 (dlina 170)
```

```
Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 2
do 3: 3
do 4: 10
do 5: 5
do 6: 14
do 7: 24
do 8: 42
do 9: 9
do 10: 22
do 11: 36
do 12: 58
do 13: 70
do 14: 98
do 15: 129
do 16: 170
```

```
=== ALGORITHM FLOYD-WARSHALL ===
Matritsa krachaishih rasstoyaniy:
1: 0 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
2: 2 0 5 8 7 14 24 42 40 22 36 58 70 98 129 170
3: 3 5 0 12 8 12 21 44 12 25 33 60 73 101 126 172
4: 10 8 12 0 15 20 33 32 19 28 45 48 80 104 138 160
5: 5 7 8 15 0 19 29 47 14 27 41 63 65 103 134 175
6: 14 12 17 20 19 0 38 48 23 32 50 68 84 143 196 240
7: 24 26 21 33 29 38 0 56 33 46 54 81 94 122 165 184
8: 42 40 44 32 47 48 56 0 51 60 77 80 112 132 161 178
9: 9 11 12 19 14 23 33 31 0 31 45 67 79 107 130 139
10: 22 20 25 28 27 32 46 45 45 0 58 76 92 116 145 188
11: 36 33 33 45 54 54 77 77 88 58 0 93 106 134 159 205
12: 58 56 60 48 63 68 81 88 92 76 92 0 128 152 186 240
13: 70 72 73 80 65 84 94 112 99 116 134 128 0 168 195 240
14: 98 96 101 104 103 84 122 132 107 116 134 152 168 0 227 244
15: 129 131 126 138 134 143 161 161 151 159 186 195 227 240 0 240
16: 170 168 172 160 175 176 184 178 179 188 205 240 240 224 240 0
```

Tect13 – Herschel_11vershin



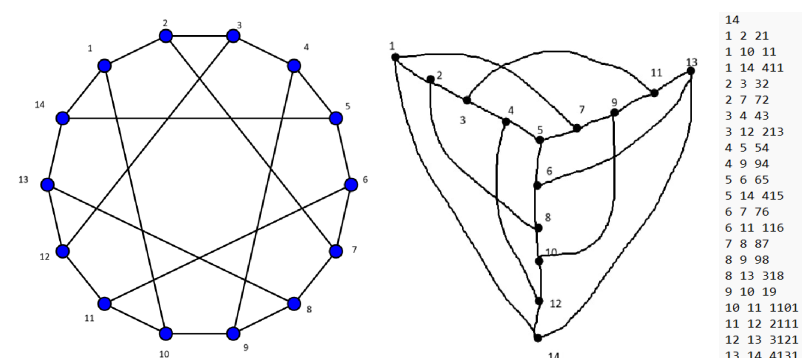
```
Fayl: tests/test13.txt
Vershin: 11, ryober: 18

=== ALGORITHM DEYKSTRY (ot vershiny 1) ===
V grafe est otricatelnye rebra.
Algoritm Deykstry ne primenim 4 propuskaem.
```

```
Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: -7856523
do 3: -2242106990
do 4: 27514402
do 5: -8570574
do 6: 19727977730
do 7: -38860933
do 8: -7856434
do 9: -38860195
do 10: -38859735
do 11: -13989987
```

```
=== ALGORITHM FLOYD-WARSHALL ===
Matritsa krachaishih rasstoyaniy:
1: 0 -7856523 -2242106990 27514402 -8570574 19727977730 -38860933 -7856434 -38860195 -38859735 -13989987
2: 9918006823 0 6125148378 9890492421 -714051 28095233098 -31004410 89 -31003672 -31003212 -6133464
3: 16044583303 6918007114 0 16017068901 6125862429 21970084720 6095572870 6918007025 6095572808 6095573268 6911873650
4: 27514402 19657879 -2214592588 0 18943828 0 18943828 0 18943828 0 18943828
5: 9918720874 792144685 6125862429 9891206472 746897672 0 746897672 0 746897672 0 746897672
6: 9874187912 747611723 6872760101 9846673510 821720993 28917668142 821720993 0 821720993 0 821720993
7: 9919011223 8224356044 6947583422 9921496831 -713962 28917668880 28917668880 0 28917668880 0 28917668880
8: 9918006912 89 6125148467 9890492510 -31004321 0 -31004321 0 -31004321 0 -31004321
9: 9919010495 822435782 6947584160 9921496093 821721731 28917668880 28917668880 738 822435693 0 822435693
10: 9919010955 822436242 6947584620 9921496553 821722191 28917669340 28917669340 1198 822436153 460 822436153
11: 533934678764 524808104051 530933252429 533907164362 524807390000 552903337149 523985669007 524808103962 523985668269 523985667809 0
```

Tect14 – Hivud_14vershin



```
Fayl: tests/test14.txt
Vershin: 14, ryober: 21

=== ALGORITM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 21
do 3: 53
do 4: 96
do 5: 150
do 6: 169
do 7: 93
do 8: 128
do 9: 30
do 10: 11
do 11: 285
do 12: 266
do 13: 446
do 14: 411
```

```
Krachaishie puti ot 1:
do 2: 1 -> 2 (dlina 21)
do 3: 1 -> 2 -> 3 (dlina 53)
do 4: 1 -> 2 -> 3 -> 4 (dlina 96)
do 5: 1 -> 2 -> 3 -> 4 -> 5 (dlina 150)
do 6: 1 -> 2 -> 7 -> 6 (dlina 169)
do 7: 1 -> 2 -> 7 (dlina 93)
do 8: 1 -> 10 -> 9 -> 8 (dlina 128)
do 9: 1 -> 10 -> 9 (dlina 30)
do 10: 1 -> 10 (dlina 11)
do 11: 1 -> 2 -> 7 -> 6 -> 11 (dlina 285)
do 12: 1 -> 2 -> 3 -> 12 (dlina 266)
do 13: 1 -> 10 -> 9 -> 8 -> 13 (dlina 446)
do 14: 1 -> 14 (dlina 411)
```

```
Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 21
do 3: 53
do 4: 96
do 5: 150
do 6: 169
do 7: 93
do 8: 128
do 9: 30
do 10: 11
do 11: 285
do 12: 266
do 13: 446
do 14: 411
```

```
=== ALGORITM FLOYD-WARSHALL ===
Matritsa krachaishih rasstoyaniy:
      1      2      3      4      5      6      7      8      9      10      11      12      13      14
1: 0      21      53      96      150      169      93      128      30      11      285      266      446      411
2: 21      0      32      75      129      148      72      149      51      32      264      245      467      432
3: 53      32      0      43      97      162      104      191      82      64      278      213      499      464
4: 96      75      43      0      54      119      147      192      94      107      235      256      510      469
5: 150      129      97      54      0      65      141      228      148      161      181      310      546      415
6: 169      148      162      119      65      0      76      163      199      180      116      375      481      480
7: 93      72      104      147      141      76      0      87      123      104      192      317      485      504
8: 128      149      181      192      228      163      87      0      98      117      279      394      318      539
9: 30      51      83      94      148      199      123      98      0      19      315      296      416      441
10: 11      32      64      187      161      180      104      117      19      0      296      277      435      422
11: 285      264      278      235      181      116      192      279      315      296      0      491      597      596
12: 266      245      213      256      310      375      317      394      296      277      491      0      712      677
13: 446      467      499      518      546      481      485      318      416      435      597      712      0      857
14: 411      432      464      469      415      480      504      539      441      422      596      677      857      0
```

Тест15 – 1024vershin

```
Fayl: tests/test15.txt
Vershin: 1024, ryober: 1954

=== ALGORITM DEYKSTRY (ot vershiny 1) ===
Krachaishie rasstoyaniya ot 1:
do 1: 0
do 2: 3
do 3: 8
do 4: 15
do 5: 24
do 6: 35
do 7: 48
do 8: 63
do 9: 80
do 10: 99
```

```
do 289: 1 -> 33 -> 65 -> 97 -> 129 -> 161 -> 193 -> 225 -> 257 -> 289 (dlina 2610)
do 290: 1 -> 2 -> 34 -> 66 -> 98 -> 130 -> 162 -> 194 -> 226 -> 258 -> 290 (dlina 2631)
do 291: 1 -> 2 -> 3 -> 35 -> 67 -> 99 -> 131 -> 163 -> 195 -> 227 -> 259 -> 291 (dlina 2654)
do 292: 1 -> 2 -> 3 -> 4 -> 36 -> 68 -> 100 -> 132 -> 164 -> 196 -> 228 -> 260 -> 292 (dlina 2679)
do 293: 1 -> 2 -> 3 -> 4 -> 5 -> 37 -> 69 -> 101 -> 133 -> 165 -> 197 -> 229 -> 261 -> 293 (dlina 2706)
do 294: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 38 -> 70 -> 102 -> 134 -> 166 -> 198 -> 230 -> 262 -> 294 (dlina 2735)
do 295: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 39 -> 71 -> 103 -> 135 -> 167 -> 199 -> 231 -> 263 -> 295 (dlina 2766)
do 296: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 40 -> 72 -> 104 -> 136 -> 168 -> 200 -> 232 -> 264 -> 296 (dlina 2799)
do 297: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 41 -> 73 -> 105 -> 137 -> 169 -> 201 -> 233 -> 265 -> 297 (dlina 2834)
do 298: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 42 -> 74 -> 106 -> 138 -> 170 -> 202 -> 234 -> 266 -> 298 (dlina 2871)
do 299: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 43 -> 75 -> 107 -> 139 -> 171 -> 203 -> 235 -> 267 -> 299 (dlina 2910)
do 300: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 44 -> 76 -> 108 -> 140 -> 172 -> 204 -> 236 -> 268 -> 300 (dlina 2954)
```

```
12221      10184      8145
      42567      40560      38551
      10215      8176

Krachaishie rasstoyaniya ot 1 (Floyd):
do 1: 0
do 2: 3
do 3: 8
do 4: 15
do 5: 24
do 6: 35
do 7: 48
do 8: 63
do 9: 80
do 10: 99
do 11: 120
do 12: 143
do 13: 168
```

Вывод: реализована программа на С для поиска кратчайших путей в взвешенном графе алгоритмами Дейкстры и Флойда–Уоршелла. Проверена на 15 тестах — от малых графов до графа с 1024 вершинами. Результаты совпали с теорией.